
Problem 1

Use the wbca data(copy and paste the following command into R console:library(faraway);data("wbca"). The dataset wbca comes from a study of breast cancer in Wisconsin. There are 681 cases of potentially cancerous tumors of which 238 are actually malignant. Determining whether a tumor is really malignant is traditionally determined by an invasive surgical procedure. The purpose of this study was to determine whether a new procedure called fine needle aspiration, which draws only a small sample of tissue, could be effective in determining tumor status. Use 70% of the data for train data set and use remaining data (30% of data) for test data set (use set.seed(1023)). (50 points, 10 points each)

a-) Fit a binary regression with Class as the response and the other nine variables as predictors on the train data set.

Report the residual deviance and associated degrees of freedom. Can this information be used to determine if this model fits the data? Explain.

Ans: Residual Deviance: 52.463 Residual Degrees of Freedom (DF): 467

The residual deviance (52.463 on 467 df) is much smaller than the null deviance (620.164 on 476 df).

This large reduction in deviance indicates that the predictors provide a robust improvement in logistic Regression model fit. Because the residual deviance is small relative to its degrees of freedom, there is no evidence of lack of fit.

Therefore, based on deviance alone, it's safe to say that the logistic regression model fits the data very well.

```
# Wisconsin breast cancer database
library(faraway)
data("wbca")
#help("wbca")
#View(wbca)

library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
##
```

```
## Attaching package: 'lattice'
```

```
## The following object is masked from 'package:faraway':
```

```
##
```

```
##      melanoma
```

```
#Splitting the data into training and testing samples:
```

```
set.seed(1023)
```

```
#Do 70/30 split, 70 for Train Data and 30 for Test Data.
```

```
DataSplit = createDataPartition( y = wbca$Class, p = 0.7, list = FALSE )
```

```
#Creating test and train data set
```

```
TrainData = wbca[DataSplit, ]
```

```
TestData = wbca[-DataSplit, ]
```

```
#View(TrainData)
```

```
#Checking if you have 70% test and 30% train
```

```

Num_observations = nrow(wbca)
Num_observations_train = nrow(TrainData)
Num_observations_test = nrow(TestData)

print(Num_observations)

## [1] 681

print(Num_observations_train)

## [1] 477

print(Num_observations_test)

## [1] 204

#Logistic Regression
Binary_model = glm(Class ~ . , data = TrainData, family = binomial)
summary(Binary_model)

##
## Call:
## glm(formula = Class ~ . , family = binomial, data = TrainData)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  13.3483     2.2789   5.857  4.7e-09 ***
## Adhes        -0.2428     0.1638  -1.483  0.138133
## BNucl        -0.5141     0.1421  -3.618  0.000297 ***
## Chrom       -0.9182     0.3049  -3.012  0.002599 **
## Epith       -0.1705     0.2292  -0.744  0.457071
## Mitos       -0.3832     0.3918  -0.978  0.327977
## NNucl       -0.2743     0.1607  -1.707  0.087905 .
## Thick       -0.9292     0.2454  -3.787  0.000152 ***
## UShap       -0.3850     0.3601  -1.069  0.284980
## USize        0.4060     0.3563   1.140  0.254459
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 620.164  on 476  degrees of freedom
## Residual deviance:  52.463  on 467  degrees of freedom
## AIC: 72.463
##
## Number of Fisher Scoring iterations: 9

# CUSTOM EQUATION PRINTER
dispRegFunc <- function(Binary_model)
{
  coefs <- Binary_model$coefficients
  b0 <- coefs[1]
  n <- length(coefs)
  eq <- paste0("Y = ", round(b0, 6))
  for (i in 2:n) {
    sign <- ifelse(coefs[i] >= 0, " + ", " - ")

```

```

    eq <- paste0(eq, sign, abs(round(coefs[i], 6)), ".*", names(coefs)[i])
  }
  eq
}

```

```
dispRegFunc(Binary_model)
```

```
## [1] "Y = 13.348345 - 0.242836*Adhes - 0.514064*BNucl - 0.918155*Chrom - 0.17046*Epith - 0.383217*Mit
```

b-) Use AIC as the criterion to determine the best subset of variables. Perform Hosmer-Lemeshow Goodness of Fit Test and comment (Use the step function.)

Ans:

We fail to reject the null hypothesis that the model fits the data well, because the Hosmer-Lemeshow test has a high p-value ($0.7322 > 0.05$)

This result means:

1. There is no evidence of lack of fit.
2. The predicted probabilities align well with the observed classifications.
3. The reduced model from stepwise selection appears adequate and well-fitting.

The stepwise AIC procedure successfully removed unnecessary variables, and the Hosmer-Lemeshow test confirms that the reduced logistic regression model fits the data well.

```
# AIC-based stepwise selection
```

```
Binary_model.Step.Func = step(Binary_model, trace = 0)
summary(Binary_model.Step.Func)
```

```
##
## Call:
## glm(formula = Class ~ Adhes + BNucl + Chrom + NNucl + Thick,
##      family = binomial, data = TrainData)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  12.6229      1.9815   6.370 1.89e-10 ***
## Adhes        -0.2748      0.1603  -1.714  0.08650 .
## BNucl        -0.5196      0.1291  -4.025 5.69e-05 ***
## Chrom       -0.8366      0.2719  -3.077  0.00209 **
## NNucl       -0.3353      0.1253  -2.677  0.00744 **
## Thick       -1.0059      0.2225  -4.521 6.16e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 620.164  on 476  degrees of freedom
## Residual deviance:  56.117  on 471  degrees of freedom
## AIC: 68.117
##
## Number of Fisher Scoring iterations: 9

```

```
### Hosmer-Lemeshow Goodness of Fit Test
```

```
library(ResourceSelection)
```

```
## ResourceSelection 0.3-6    2023-06-27
```

```
hoslem.test(Binary_model.Step.Func$y, fitted(Binary_model.Step.Func), g=5)
```

```
##
```

```
## Hosmer and Lemeshow goodness of fit (GOF) test
```

```
##
```

```
## data: Binary_model.Step.Func$y, fitted(Binary_model.Step.Func)
```

```
## X-squared = 1.2871, df = 3, p-value = 0.7322
```

c-) Suppose that a cancer is classified as benign if $p > 0.5$ and malignant if $p < 0.5$. Compute the confusion matrix and accuracy rates if this method is applied to the train data with the reduced model.

Ans:

We will drop the variable Adhes, since $p > 0.05$. The new logistic regression model will be Binary_model.significant_pValues.

The key performance metrics for the model on the training data are:

Accuracy = 0.9706, meaning the model correctly classifies about 97% of tumors.

Only 7 benign cases were misclassified as malignant, and 7 malignant cases were misclassified as benign, showing very balanced error rates.

Math calculation from slides.

Sensitivity (Recall for Class = 1): 0.9773 (The rate of correctly identifying Benign tumors)

Specificity (Recall for Class = 0): 0.9586 (The rate of correctly identifying Malignant tumors))

Precision: 0.9773 (The proportion of predicted Benign tumors that were truly Benign)

F1 Score: 0.9773

The F1 score of 0.9773 indicates that the reduced logistic regression model has a great balance between precision and recall, meaning it consistently identifies benign tumors with both high accuracy and reliability.

```
library(caret)
```

```
# logistic regression with significant p-values
```

```
Binary_model.significant_pValues = glm(Class ~ BNucl + Chrom + NNucl + Thick,  
    family = binomial, data = TrainData)
```

```
# Predicted probabilities on the TRAIN data
```

```
predicted_probabilities = predict(Binary_model.significant_pValues, TrainData, type="response")
```

```
# Confusion Matrix
```

```
true_labels = TrainData$Class
```

```
# using 0.5 as a cutoff to predict the class
```

```
predictions = ifelse(predicted_probabilities > 0.5, 1, 0)
```

```
confusion_matrix <- confusionMatrix(as.factor(predictions), as.factor(true_labels), mode="prec_recall",
```

```
#positive = 1 bc 1 is benign
```

```
confusion_matrix
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction    0    1
```

```
##           0 162    7
```

```
##           1   7 301
```

```
##
##           Accuracy : 0.9706
##           95% CI : (0.9512, 0.9839)
##    No Information Rate : 0.6457
##    P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9359
##
## Mcnemar's Test P-Value : 1
##
##           Precision : 0.9773
##           Recall : 0.9773
##           F1 : 0.9773
##           Prevalence : 0.6457
##    Detection Rate : 0.6310
##    Detection Prevalence : 0.6457
##    Balanced Accuracy : 0.9679
##
##    'Positive' Class : 1
##
```

```
#help("wbca")
```

d-) Suppose we change the cutoff to 0.9 so that $p < 0.9$ is classified as malignant and $p > 0.9$ as benign. Compute the confusion matrix.

Ans:

Accuracy: 0.9748 —> slightly higher than the 0.5-cutoff model.

Benign misclassified as malignant: 11 cases

Malignant misclassified as benign: only 1 case (a major improvement).

Sensitivity (Recall for Class = 1): 0.9643 This is slightly lower than before —> fewer benign tumors correctly identified.

Specificity (Recall for Class = 0): 0.9941 This is extremely high —> very few malignant tumors misclassified.

Precision: 0.9966 Nearly every predicted benign tumor is actually benign.

F1 Score: 0.9802 Shows excellent balance between precision and recall despite the stricter cutoff.

```
# Predicted probabilities on the TRAIN data
predicted_probabilities = predict(Binary_model.significant_pValues, TrainData, type="response")

# Confusion Matrix
true_labels = TrainData$Class

# using 0.9 as a cutoff to predict the class
predictions_Train_0.9 = ifelse(predicted_probabilities > 0.9, 1, 0)
confusion_matrix_Train_0.9 = confusionMatrix(as.factor(predictions_Train_0.9), as.factor(true_labels),

#positive = 1 bc 1 is benign
confusion_matrix_Train_0.9
```

```
## Confusion Matrix and Statistics
##
##           Reference
```

```

## Prediction    0    1
##              0 168  11
##              1   1 297
##
##              Accuracy : 0.9748
##              95% CI : (0.9565, 0.9869)
##      No Information Rate : 0.6457
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.9457
##
##      McNemar's Test P-Value : 0.009375
##
##              Precision : 0.9966
##              Recall : 0.9643
##              F1 : 0.9802
##              Prevalence : 0.6457
##      Detection Rate : 0.6226
##      Detection Prevalence : 0.6247
##      Balanced Accuracy : 0.9792
##
##      'Positive' Class : 1
##

```

e-) Repeat part c and d on the test data set and comment on the model performance and which cutoff will you choose?

Ans:

Using the 0.5 cutoff on the test data:

Accuracy = 0.9559 (95.6%)

9 total misclassifications: 4 False Negatives (Predicted Malignant, was Benign) and 5 False Positive (Predicted Benign, was Malignant).

Sensitivity (Recall for class 1 / Benign) = 0.9704

Specificity (Recall for class 0 / Malignant) = 0.9275

F1 Score = 0.9668

Using the 0.9 cutoff on the test data:

Accuracy = 0.9510 (very similar but slightly reduced)

10 total misclassifications: 6 False Negatives (Predicted Malignant, was Benign) and 4 False False Positives (Predicted Benign, was Malignant).

Sensitivity (Benign recall) decreases slightly to 0.9556

Specificity (Malignant recall) increases slightly to 0.9420

F1 Score = 0.9627 (slightly lower than 0.5 cutoff)

The model generalizes extremely well to the unseen Test Data, maintaining an Accuracy above 95% in both cases. Given the results, the 0.5 cutoff performs better overall, with the highest F1 score and balanced accuracy. Therefore, I would choose the 0.5 cutoff for this dataset.

```

##### TEST DATA SET
##### 0.5 cutoff:

```

```

library(caret)
# logistic regression with significant p-values
Binary_model.significant_pValues = glm(Class ~ BNucl + Chrom + NNucl + Thick,
    family = binomial, data = TrainData)

# Predicted probabilities on the TEST data
predicted_probabilities = predict(Binary_model.significant_pValues, TestData, type="response")

####Confusion Matrix####

true_labels = TestData$Class

# using 0.5 as a cutoff to predict the class
predictions_0.5 = ifelse(predicted_probabilities > 0.5, 1, 0)

confusion_matrix_0.5 = confusionMatrix(as.factor(predictions_0.5),
as.factor(true_labels), mode="prec_recall", positive = "1")

#positive = 1 bc 1 is benign
confusion_matrix_0.5

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0  64   4
##           1   5 131
##
##           Accuracy : 0.9559
##           95% CI : (0.9179, 0.9796)
##    No Information Rate : 0.6618
##    P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9011
##
##  McNemar's Test P-Value : 1
##
##           Precision : 0.9632
##           Recall : 0.9704
##           F1 : 0.9668
##           Prevalence : 0.6618
##    Detection Rate : 0.6422
##    Detection Prevalence : 0.6667
##    Balanced Accuracy : 0.9490
##
##    'Positive' Class : 1
##

#### 0.9 cutoff:

# Confusion Matrix
# using 0.9 as a cutoff to predict the class
predictions_0.9 = ifelse(predicted_probabilities > 0.9, 1, 0)

```

```

confusion_matrix_0.9 = confusionMatrix(as.factor(predictions_0.9), as.factor(true_labels), mode="prec_r
#positive = 1 bc 1 is benign
cat("\n\n\n -----0.9 cutoff-----")

```

```

##
##
##
## -----0.9 cutoff-----

```

```

confusion_matrix_0.9

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0  65    6
##           1   4 129
##
##           Accuracy : 0.951
##           95% CI : (0.9117, 0.9762)
##       No Information Rate : 0.6618
##       P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.8913
##
##  McNemar's Test P-Value : 0.7518
##
##           Precision : 0.9699
##           Recall : 0.9556
##           F1 : 0.9627
##           Prevalence : 0.6618
##       Detection Rate : 0.6324
##   Detection Prevalence : 0.6520
##       Balanced Accuracy : 0.9488
##
##       'Positive' Class : 1
##

```

Problem 2

Use train and test data set in problem 1 to answer the questions below.(50 points)

a-) Fit a decision tree model to predict Class by using the other nine variables as predictors on the train data set by using xgboost, adaboost, random forest, boosting in library C5.0, tree and rpart libraries and methods. Calculate the confusing matrix for each method and comment on your findings. (30 points)

Ans:

1. Random Forest

Accuracy: 1.0000

Sensitivity (Recall for Class 1): 1.0000

Specificity (Recall for Class 0): 1.0000

F1 Score: 1.0000

Misclassifications: 0

This likely reflects overfitting

2. C5.0 Boosting

Accuracy: 0.9895

Sensitivity: 0.9903

Specificity: 0.9882

Misclassifications: 5 (2 false positives, 3 false negatives)

Very strong performance, slight overfitting but not extreme. Uses a small tree (size = 11).

3. Tree (tree package)

Accuracy: 0.9832

Sensitivity: 0.9935

Specificity: 0.9645

Misclassifications: 8 (6 false positives, 2 false negatives)

Simpler model than C5.0; high sensitivity but lower specificity.

4. RPART

Accuracy: 0.9665

Sensitivity: 0.9610

Specificity: 0.9763

Misclassifications: 16 (12 FN, 4 FP)

The worst classifier on the training data set.

5. XGBoost

Accuracy: 0.9958

Sensitivity: 0.9968

Specificity: 0.9941

Misclassifications: 2 (1 FN, 1 FP)

Extremely robust performance. Slightly below perfect, meaning less overfitting than Random Forest/Adaboost but there is some indication of overfitting.

6. AdaBoost

Accuracy: 1.0000

Sensitivity: 1.0000

Specificity: 1.0000

Misclassifications: 0

Also perfectly fits the training data, which suggests overfitting.

Conclusion: The boosting-type models (XGBoost, C5.0, AdaBoost) perform the best, while RPART is the weakest/worst classifier. Models with perfect accuracy (Random Forest, AdaBoost) suggest strong indication of overfitting.

```

set.seed(1023)
TrainData$Class = factor(TrainData$Class, levels = c("0", "1"))
#### Random Forest / Bagging
library(randomForest)

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##     margin

# install.packages("randomForest")
# Recall that bagging is simply a special case of a random forest with m=p
cat("\n\n--- RandomForest ---\n") # for printing

##
##
## --- RandomForest ---

random.forest.wcb = randomForest(Class ~ ., data = TrainData, mtry = 9,
                                importance = TRUE)
random.forest.wcb

##
## Call:
## randomForest(formula = Class ~ ., data = TrainData, mtry = 9,      importance = TRUE)
##              Type of random forest: classification
##              Number of trees: 500
## No. of variables tried at each split: 9
##
##          OOB estimate of  error rate: 3.35%
## Confusion matrix:
##      0   1 class.error
## 0 160   9  0.05325444
## 1   7 301  0.02272727

# **Confusion Matrix:**
library(caret)
random.forest.prediction.TrainData = predict(random.forest.wcb, newdata = TrainData )

#Same factor levels for the error:
pred.randomforest = factor(random.forest.prediction.TrainData, levels = c("0","1"))
true.randomforest = factor(TrainData$Class, levels = c("0","1"))

Confu.m.rf = confusionMatrix(pred.randomforest, true.randomforest, positive = "1")
Confu.m.rf

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##      0 169    0
##      1   0 308

```

```

##
##          Accuracy : 1
##          95% CI : (0.9923, 1)
##    No Information Rate : 0.6457
##    P-Value [Acc > NIR] : < 2.2e-16
##
##          Kappa : 1
##
##    Mcnemar's Test P-Value : NA
##
##          Sensitivity : 1.0000
##          Specificity : 1.0000
##    Pos Pred Value : 1.0000
##    Neg Pred Value : 1.0000
##          Prevalence : 0.6457
##    Detection Rate : 0.6457
##    Detection Prevalence : 0.6457
##    Balanced Accuracy : 1.0000
##
##    'Positive' Class : 1
##
#### Boosting with C5.0
library(C50)
C50boost.wcb.model = C5.0(x = TrainData[, -1], y = as.factor(TrainData$Class))
cat("\n\n--- C5.0 Boosting Result ---\n")

##
##
## --- C5.0 Boosting Result ---
summary(C50boost.wcb.model)

##
## Call:
## C5.0.default(x = TrainData[, -1], y = as.factor(TrainData$Class))
##
##
## C5.0 [Release 2.07 GPL Edition]          Sat Apr 25 21:33:08 2026
## -----
##
## Class specified by attribute `outcome'
##
## Read 477 cases (10 attributes) from undefined.data
##
## Decision tree:
##
## USize <= 3:
## :...BNucl <= 2: 1 (284/1)
## :   BNucl > 2:
## :     :...Thick <= 3: 1 (13)
## :     :   Thick > 3:
## :       :...Chrom > 2: 0 (25/2)
## :       :   Chrom <= 2:
## :         :...BNucl <= 4: 1 (3)

```

```

## :                BNucl > 4: 0 (3/1)
## USize > 3:
## :...Chrom > 4: 0 (106)
##   Chrom <= 4:
##     :...Thick > 6: 0 (29)
##       Thick <= 6:
##         :...BNucl > 9: 0 (5)
##           BNucl <= 9:
##             :...Chrom <= 3: 1 (4)
##               Chrom > 3:
##                 :...Adhes <= 2: 0 (2)
##                   Adhes > 2: 1 (3/1)
##
##
## Evaluation on training data (477 cases):
##
##      Decision Tree
##      -----
##      Size      Errors
##
##      11      5( 1.0%)  <<
##
##
##      (a)   (b)   <-classified as
##      ----  ----
##      167    2   (a): class 0
##      3    305  (b): class 1
##
##
## Attribute usage:
##
## 100.00% USize
##  71.70% BNucl
##  37.74% Chrom
##  18.24% Thick
##   1.05% Adhes
##
##
## Time: 0.0 secs

```

```

# **Confusion Matrix:**
library(caret)
C50.pred.train = predict(C50boost.wcb.model, newdata = TrainData)
#Same factor levels for the error:
pred.C50 = factor(C50.pred.train, levels = c("0","1"))
true.C50 = factor(TrainData$Class, levels = c("0","1"))

Confu.m.C50 = confusionMatrix(pred.C50, true.C50, positive = "1")
Confu.m.C50

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 167   3

```

```

##          1    2 305
##
##          Accuracy : 0.9895
##          95% CI : (0.9757, 0.9966)
##    No Information Rate : 0.6457
##    P-Value [Acc > NIR] : <2e-16
##
##          Kappa : 0.9771
##
##    McNemar's Test P-Value : 1
##
##          Sensitivity : 0.9903
##          Specificity : 0.9882
##    Pos Pred Value : 0.9935
##    Neg Pred Value : 0.9824
##          Prevalence : 0.6457
##    Detection Rate : 0.6394
##    Detection Prevalence : 0.6436
##    Balanced Accuracy : 0.9892
##
##    'Positive' Class : 1
##

###Tree:

library(tree)
library(caret)
TrainData$Class = factor(TrainData$Class, levels = c("0", "1"))
tree.wcb = tree(Class ~ ., data = TrainData)

cat("\n\n--- Tree Result ---\n")

##
##
## --- Tree Result ---

summary(tree.wcb)

##
## Classification tree:
## tree(formula = Class ~ ., data = TrainData)
## Variables actually used in tree construction:
## [1] "USize" "BNucl" "Epith" "Thick" "Chrom"
## Number of terminal nodes: 9
## Residual mean deviance: 0.08128 = 38.04 / 468
## Misclassification error rate: 0.01677 = 8 / 477

# **Confusion Matrix:**
tree.pred.train = predict(tree.wcb, newdata = TrainData, type = "class")

pred.tree = factor(tree.pred.train, levels = c("0", "1"))
true.tree = factor(TrainData$Class, levels = c("0", "1"))

Confu.m.tree = confusionMatrix(pred.tree, true.tree, positive = "1")
Confu.m.tree

## Confusion Matrix and Statistics

```

```

##
##           Reference
## Prediction   0    1
##           0 163   2
##           1   6 306
##
##           Accuracy : 0.9832
##           95% CI : (0.9672, 0.9927)
##           No Information Rate : 0.6457
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9631
##
## Mcnemar's Test P-Value : 0.2888
##
##           Sensitivity : 0.9935
##           Specificity : 0.9645
##           Pos Pred Value : 0.9808
##           Neg Pred Value : 0.9879
##           Prevalence : 0.6457
##           Detection Rate : 0.6415
##           Detection Prevalence : 0.6541
##           Balanced Accuracy : 0.9790
##
##           'Positive' Class : 1
##
### RPART:
cat("\n\n--- This is the RPART ---\n")

##
##
## --- This is the RPART ---

library(rpart)

##
## Attaching package: 'rpart'
## The following object is masked from 'package:faraway':
##
## solder

rpart.wcb = rpart(Class ~ ., data = TrainData, method = "class")
rpart.wcb

## n= 477
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
## 1) root 477 169 1 (0.35429769 0.64570231)
## 2) USize>=3.5 149 6 0 (0.95973154 0.04026846) *
## 3) USize< 3.5 328 26 1 (0.07926829 0.92073171)
## 6) BNucl>=5.5 18 2 0 (0.88888889 0.11111111) *
## 7) BNucl< 5.5 310 10 1 (0.03225806 0.96774194)
## 14) NNucl>=3.5 10 4 0 (0.60000000 0.40000000) *

```

```

##      15) NNucl< 3.5 300    4 1 (0.01333333 0.98666667) *
# **Confusion Matrix:**
rpart.pred.train = predict(rpart.wcb, newdata = TrainData, type = "class")

pred.rpart = factor(rpart.pred.train, levels = c("0","1"))
true.rpart = factor(TrainData$Class, levels = c("0","1"))

Confu.m.rpart = confusionMatrix(pred.rpart, true.rpart, positive = "1")
Confu.m.rpart

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 165  12
##           1   4 296
##
##           Accuracy : 0.9665
##           95% CI : (0.9461, 0.9807)
##    No Information Rate : 0.6457
##    P-Value [Acc > NIR] : < 2e-16
##
##           Kappa : 0.9275
##
##    McNemar's Test P-Value : 0.08012
##
##           Sensitivity : 0.9610
##           Specificity : 0.9763
##           Pos Pred Value : 0.9867
##           Neg Pred Value : 0.9322
##           Prevalence : 0.6457
##           Detection Rate : 0.6205
##    Detection Prevalence : 0.6289
##           Balanced Accuracy : 0.9687
##
##           'Positive' Class : 1
##
#### XGBoost:
library(xgboost)
xgb_train_y_numeric = as.numeric(as.character(TrainData$Class))
xgb_train_x = model.matrix(Class ~ . -1, data = TrainData)

xgboost.wcb = xgboost(data = xgb_train_x, label = xgb_train_y_numeric,
                      objective = "binary:logistic", nrounds = 30, verbose = 0)
cat("\n\n--- XGBoost Result ---\n")

##
##
## --- XGBoost Result ---
summary(xgboost.wcb)

##           Length Class           Mode

```

```
## handle          1  xgb.Booster.handle externalptr
## raw             37596 -none- raw
## niter           1  -none- numeric
## evaluation_log  2  data.table list
## call            14  -none- call
## params          2  -none- list
## callbacks       1  -none- list
## feature_names   9  -none- character
## nfeatures       1  -none- numeric

# **Confusion Matrix:**
xgb_pred_prob = predict(xgboost.wcb, newdata = xgb_train_x)
xgb.pred.train = ifelse(xgb_pred_prob > 0.5, 1, 0)

pred.xgb = factor(xgb.pred.train, levels = c("0", "1"))
true.xgb = factor(TrainData$Class, levels = c("0", "1"))

Confu.m.xgb = confusionMatrix(pred.xgb, true.xgb, positive = "1")
Confu.m.xgb
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 168    1
##           1    1 307
##
##           Accuracy : 0.9958
##           95% CI : (0.9849, 0.9995)
##           No Information Rate : 0.6457
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9908
##
## Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9968
##           Specificity : 0.9941
##           Pos Pred Value : 0.9968
##           Neg Pred Value : 0.9941
##           Prevalence : 0.6457
##           Detection Rate : 0.6436
##           Detection Prevalence : 0.6457
##           Balanced Accuracy : 0.9954
##
##           'Positive' Class : 1
##
```

```
### Adaboost:
library(adabag)
```

```
## Loading required package: foreach
## Loading required package: doParallel
## Loading required package: iterators
```



```
## Loading required package: parallel
## Warning in rgl.init(initValue, onlyNULL): X11 error: GLXBadContext
## Warning: 'rgl.init' failed, will use the null device.
## See '?rgl.useNULL' for ways to avoid this warning.
TrainData$Class = as.factor(TrainData$Class)
ada.boost.wcb.model = boosting(Class ~ . , data = TrainData, mfinal = 30)
cat("\n\n--- Adaboost Result ---\n")
```

```
##
##
## --- Adaboost Result ---
```

```
summary(ada.boost.wcb.model)
```

```
##           Length Class  Mode
## formula         3    formula call
## trees           30   -none- list
## weights         30   -none- numeric
## votes          954   -none- numeric
## prob            954   -none- numeric
## class           477   -none- character
## importance        9   -none- numeric
## terms            3    terms call
## call             4   -none- call
```

```
# **Confusion Matrix:**
```

```
ada.train.pred = predict(ada.boost.wcb.model, newdata = TrainData)
```

```
pred.ada = factor(ada.train.pred$class, levels = c("0","1"))
```

```
true.ada = factor(TrainData$Class, levels = c("0","1"))
```

```
Confu.m.ada = confusionMatrix(pred.ada, true.ada, positive = "1")
```

```
Confu.m.ada
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction  0    1
```

```
##           0 169    0
```

```
##           1   0 308
```

```
##
```

```
##           Accuracy : 1
```

```
##           95% CI : (0.9923, 1)
```

```
##           No Information Rate : 0.6457
```

```
##           P-Value [Acc > NIR] : < 2.2e-16
```

```
##
```

```
##           Kappa : 1
```

```
##
```

```
##           McNemar's Test P-Value : NA
```

```
##
```

```
##           Sensitivity : 1.0000
```

```
##           Specificity : 1.0000
```

```
##           Pos Pred Value : 1.0000
```

```
##           Neg Pred Value : 1.0000
```

```
##           Prevalence : 0.6457
##           Detection Rate : 0.6457
##      Detection Prevalence : 0.6457
##           Balanced Accuracy : 1.0000
##
##           'Positive' Class : 1
##
```

b-) Calculate the confusing matrices on the test data set and compare against the logistic regression model confusing matrix. Which model would you choose? (20 points)

Ans:

1. Logistic Regression Accuracy: 0.9559

Sensitivity (Recall for Class 1): 0.9704

Specificity (Recall for Class 0): 0.9275

Misclassifications (Total): 9 (5 False Positives, 4 False Negatives)

Very strong performance, showing a high ability to generalize despite being a linear model. Similar performance to Random Forest.

2. Random Forest Accuracy: 0.9608

Sensitivity (Recall for Class 1): 0.9778

Specificity (Recall for Class 0): 0.9275

Misclassifications (Total): 8 (5 False Positives, 3 False Negatives)

Excellent test performance, slight improvement over logistic regression-strong generalization. Very low indication of overfitting.

3. C5.0 Boosting

Accuracy: 0.9412

Sensitivity: 0.9778

Specificity: 0.8696

Misclassifications (Total): 12

9 False Positives

3 False Negatives

Higher false positives reduce specificity. Worse than RF and XGBoost on this dataset.

4. Tree

Accuracy: 0.9412

Sensitivity: 0.9778

Specificity: 0.8696

Misclassifications (Total): 12

9 False Positives

3 False Negatives

Simple model with decent sensitivity, but lowest specificity. Same results as C5.0 Boosting test performance.

5. RPART

Accuracy: 0.9608

Sensitivity: 0.9556

Specificity: 0.9710

Misclassifications (Total): 8

6 False Negatives

2 False Positives

Surprisingly strong considering how bad it was with the training data set. Very balanced and achieved the highest Specificity.

6. XGBoost

Accuracy: 0.9657

Sensitivity: 0.9778

Specificity: 0.9420

Misclassifications (Total): 7

4 False Positives

3 False Negatives

Highest overall accuracy on the test set. Highly robust and effective, providing a strong balance between Sensitivity and Specificity.

7. AdaBoost

Accuracy: 0.9657

Sensitivity: 0.9778

Specificity: 0.9420

Misclassifications (Total): 7

4 False Positives

3 False Negatives

Exactly identical to XGBoost Matches XGBoost; very strong generalization and low error.

The Champion:

I would choose the XGBoost Classifier model because it has the highest accuracy of 96.57%, the misclassification rate is also very low (7 total errors), and it maintains robust sensitivity 0.9778 and specificity 0.9420. This classifier provides the highest confidence in making correct predictions across the board.

```
##### TEST DATA SET
##### 0.5 cutoff:
library(caret)

### Logistic regression with significant p-values
Binary_model.significant_pValues = glm(Class ~ BNucl + Chrom + NNucl + Thick,
    family = binomial, data = TrainData)

# Predicted probabilities on the TEST data
predicted_probabilities = predict(Binary_model.significant_pValues, TestData, type="response")

cat("\n\n-----Logistic regression Result ----- \n")
```

```

##
##
## -----Logistic regression Result -----
##### Confusion Matrix #####
true_labels = TestData$Class

# using 0.5 as a cutoff to predict the class
predictions_0.5 = ifelse(predicted_probabilities > 0.5, 1, 0)

confusion_matrix_0.5 = confusionMatrix(as.factor(predictions_0.5),
as.factor(true_labels), mode="prec_recall", positive = "1")

#positive = 1 bc 1 is benign
confusion_matrix_0.5

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0  64    4
##           1    5 131
##
##           Accuracy : 0.9559
##           95% CI : (0.9179, 0.9796)
##    No Information Rate : 0.6618
##    P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9011
##
## Mcnemar's Test P-Value : 1
##
##           Precision : 0.9632
##           Recall : 0.9704
##           F1 : 0.9668
##           Prevalence : 0.6618
##    Detection Rate : 0.6422
##    Detection Prevalence : 0.6667
##    Balanced Accuracy : 0.9490
##
##    'Positive' Class : 1
##
##### To make sure TestData$Class has the same factor levels #####
TestData$Class = factor(TestData$Class, levels = c("0","1"))

### 1) Random Forest on TEST data -----
# predicted classes
random.forest.prediction.TestData = predict(random.forest.wcb,
newdata = TestData)

cat("\n\n-----Random Forest Result -----")

##
##
## -----Random Forest Result -----

```

```

# same factor levels
pred.randomforest.TestData = factor(random.forest.prediction.TestData,
                                     levels = c("0", "1"))
true.randomforest.TestData = factor(TestData$Class, levels = c("0", "1"))

Confu.m.rf.TestData = confusionMatrix(pred.randomforest.TestData,
                                     true.randomforest.TestData,
                                     positive = "1")

Confu.m.rf.TestData

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0  64    3
##           1   5  132
##
##           Accuracy : 0.9608
##           95% CI : (0.9242, 0.9829)
##       No Information Rate : 0.6618
##       P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9118
##
##  McNemar's Test P-Value : 0.7237
##
##           Sensitivity : 0.9778
##           Specificity : 0.9275
##           Pos Pred Value : 0.9635
##           Neg Pred Value : 0.9552
##           Prevalence : 0.6618
##           Detection Rate : 0.6471
##       Detection Prevalence : 0.6716
##           Balanced Accuracy : 0.9527
##
##           'Positive' Class : 1
##

```

```

### 2) C5.0 Boosting on TEST data -----
# predicted classes
C50.pred.TestData = predict(C50boost.wcb.model, newdata = TestData)

pred.C50.TestData = factor(C50.pred.TestData, levels = c("0", "1"))
true.C50.TestData = factor(TestData$Class, levels = c("0", "1"))

cat("\n\n-----C5.0 Boosting Result ----- \n")

```

```

##
##
## -----C5.0 Boosting Result -----
Confu.m.C50.TestData = confusionMatrix(
  pred.C50.TestData,
  true.C50.TestData,
  positive = "1"
)

```

```

)
Confu.m.C50.TestData

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0  60   3
##           1   9 132
##
##           Accuracy : 0.9412
##           95% CI : (0.8995, 0.9692)
##       No Information Rate : 0.6618
##       P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.8657
##
## Mcnemar's Test P-Value : 0.1489
##
##           Sensitivity : 0.9778
##           Specificity : 0.8696
##       Pos Pred Value : 0.9362
##       Neg Pred Value : 0.9524
##           Prevalence : 0.6618
##       Detection Rate : 0.6471
##       Detection Prevalence : 0.6912
##       Balanced Accuracy : 0.9237
##
##       'Positive' Class : 1
##

### 3) Tree on TEST data -----
tree.pred.TestData <- predict(tree.wcb, newdata = TestData, type = "class")

pred.tree.TestData = factor(tree.pred.TestData, levels = c("0","1"))
true.tree.TestData = factor(TestData$Class, levels = c("0","1"))

cat("\n\n-----Tree Result -----")

##
##
## -----Tree Result -----

Confu.m.tree.TestData = confusionMatrix(
  pred.tree.TestData,
  true.tree.TestData,
  positive = "1")

Confu.m.tree.TestData

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0  60   3
##           1   9 132

```

```

##
##          Accuracy : 0.9412
##          95% CI : (0.8995, 0.9692)
##    No Information Rate : 0.6618
##    P-Value [Acc > NIR] : <2e-16
##
##          Kappa : 0.8657
##
## Mcnemar's Test P-Value : 0.1489
##
##          Sensitivity : 0.9778
##          Specificity : 0.8696
##    Pos Pred Value : 0.9362
##    Neg Pred Value : 0.9524
##          Prevalence : 0.6618
##    Detection Rate : 0.6471
##    Detection Prevalence : 0.6912
##    Balanced Accuracy : 0.9237
##
##    'Positive' Class : 1
##

### 4) RPART on TEST data -----
rpart.pred.TestData = predict(rpart.wcb, newdata = TestData, type = "class")

pred.rpart.TestData = factor(rpart.pred.TestData, levels = c("0","1"))
true.rpart.TestData = factor(TestData$Class, levels = c("0","1"))

cat("\n\n-----RPART Result ----- \n")

##
##
## -----RPART Result -----

Confu.m.rpart.TestData = confusionMatrix(
  pred.rpart.TestData,
  true.rpart.TestData,
  positive = "1")

Confu.m.rpart.TestData

## Confusion Matrix and Statistics
##
##          Reference
## Prediction  0   1
##          0  67   6
##          1   2 129
##
##          Accuracy : 0.9608
##          95% CI : (0.9242, 0.9829)
##    No Information Rate : 0.6618
##    P-Value [Acc > NIR] : <2e-16
##
##          Kappa : 0.9136
##

```

```
## McNemar's Test P-Value : 0.2888
##
##          Sensitivity : 0.9556
##          Specificity : 0.9710
##          Pos Pred Value : 0.9847
##          Neg Pred Value : 0.9178
##          Prevalence : 0.6618
##          Detection Rate : 0.6324
##          Detection Prevalence : 0.6422
##          Balanced Accuracy : 0.9633
##
##          'Positive' Class : 1
##
```

```
### 5) XGBoost on TEST data -----
```

```
# building test matrix with the same formula
xgb_test_x = model.matrix(Class ~ . - 1, data = TestData)

# predict probabilities, then threshold at 0.5
xgb_pred_prob.TestData = predict(xgboost.wcb, newdata = xgb_test_x)
xgb.pred.TestData = ifelse(xgb_pred_prob.TestData > 0.5, 1, 0)

pred.xgb.TestData = factor(xgb.pred.TestData, levels = c("0","1"))
true.xgb.TestData = factor(TestData$Class, levels = c("0","1"))

cat("\n\n-----XGBoost Result ----- \n")
```

```
##
##
## -----XGBoost Result -----
```

```
Confu.m.xgb.TestData = confusionMatrix(
  pred.xgb.TestData,
  true.xgb.TestData,
  positive = "1"
)
Confu.m.xgb.TestData
```

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction    0    1
##          0   65    3
##          1    4  132
##
##          Accuracy : 0.9657
##          95% CI : (0.9306, 0.9861)
##          No Information Rate : 0.6618
##          P-Value [Acc > NIR] : <2e-16
##
##          Kappa : 0.9231
##
## McNemar's Test P-Value : 1
##
```



```

##           Sensitivity : 0.9778
##           Specificity : 0.9420
##           Pos Pred Value : 0.9706
##           Neg Pred Value : 0.9559
##           Prevalence : 0.6618
##           Detection Rate : 0.6471
##           Detection Prevalence : 0.6667
##           Balanced Accuracy : 0.9599
##
##           'Positive' Class : 1
##

### 6) AdaBoost on TEST data -----
ada.TestData.pred = predict(ada.boost.wcb.model, newdata = TestData)

pred.ada.TestData = factor(ada.TestData.pred$class, levels = c("0","1"))
true.ada.TestData = factor(TestData$Class, levels = c("0","1"))

cat("\n\n-----AdaBoost Result ----- \n")

##
##
## -----AdaBoost Result -----

Confu.m.ada.TestData = confusionMatrix(
  pred.ada.TestData,
  true.ada.TestData,
  positive = "1"
)
Confu.m.ada.TestData

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0   65    3
##           1    4  132
##
##           Accuracy : 0.9657
##           95% CI : (0.9306, 0.9861)
##           No Information Rate : 0.6618
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9231
##
## Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9778
##           Specificity : 0.9420
##           Pos Pred Value : 0.9706
##           Neg Pred Value : 0.9559
##           Prevalence : 0.6618
##           Detection Rate : 0.6471
##           Detection Prevalence : 0.6667
##           Balanced Accuracy : 0.9599
##

```

```
##      'Positive' Class : 1
##
```